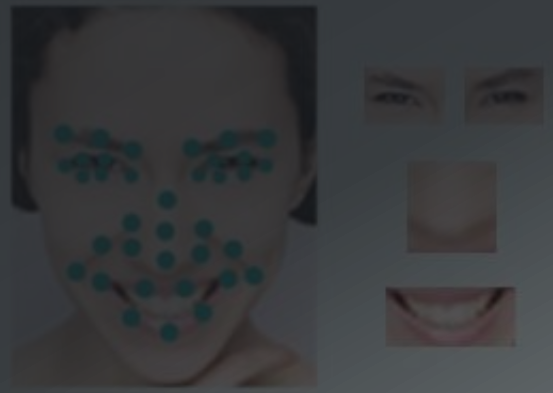
FACE EMOTION RECOGNITION

人脸情绪识别实训项目总结

基于飞桨深度学习框架的综合实践





Happy
Sad
Angry
Neutral
Bored
Contempt
Confused

2026年1月 | 25学时 | 《飞桨深度学习框架及应用》

Face Detection → Preprocessing → Feature Extraction → Classification

深度学习框架
PaddlePaddle 3.2.0

核心技术
CNN + ResNet + OpenCV

汇报人：苏于天
小组：苏于天、马子欣、杨逊源、陈健辉

汇报时间
2026年1月9日

指导教师
盛建强、刘汾

CONTENTS

目录

01 **实训目的**
项目背景、根本目标、具体目标、实训时长

02 **实训工具与团队分工**
硬件环境、软件环境、开发工具、团队分工

03 **技术架构**
核心框架、关键技术、系统整体架构

04 **实训步骤与过程**
模块一至模块八的完整开发流程

05 **实训成果**
数据集成果、模型性能、系统功能、验证案例

06 **问题与不足**
关键问题、解决方案、项目不足分析

07 **学习总结与未来展望**
核心收获、能力提升、未来发展方向

测试准确率
99.53%

数据集规模
24312张(4052+20260)

开发周期
25学时

TRAINING OBJECTIVES

实训目的

项目背景

计算机视觉领域热点应用

聚焦人脸情绪识别技术，探索AI在情感计算领域的深度应用

《飞桨深度学习框架及应用》课程

核心实践环节，理论与实践深度融合

根本目标

- **深化深度学习原理理解**，从理论认知走向实践应用
- **实现知识向实际问题解决能力的转化**，培养工程实践能力
- **培养系统思维**，理解AI项目完整生命周期

具体目标

1. **掌握飞桨框架应用**，完成从数据采集到模型部署的全链路开发
2. **培养工程思维**，掌握规范化开发流程与工具链使用
3. **培养系统思维**，理解复杂技术问题的分析与解决能力

实训时长

25学时

（一周集中实践）

理论深化	实践开发	测试优化
30%	50%	20%

TOOLS & TEAM DIVISION

实训工具（环境）和团队分工

硬件环境

计算设备

配备 NVIDIA GeForce GTX 4060 独立显卡（8GB 显存），为模型训练提供关键的 CUDA 并行计算加速能力，将训练时间从数小时缩短至数十分钟

采集设备

笔记本高清外置摄像头，用于采集高质量的人脸情绪图像数据

网络环境

实验室千兆有线网络，保障大型依赖库和框架的快速下载

开发工具

- Visual Studio Code
- Git
- 知行楼 7-505
- 千兆网络

软件与环境配置

操作系统
Windows 11 专业版 64位

开发语言
Python 3.11

核心框架
飞桨（PaddlePaddle）3.2.0 GPU版（CUDA 12.6）

关键依赖库
opencv-python、flask、numpy、scikit-learn等

团队分工

- 苏 **苏于天（25%）**：模型后端开发、调试后端接口、模型集成、项目环境搭建、PPT演讲、视频拍摄
- 陈 **陈建辉（25%）**：模型训练与超参数优化、Web前端界面开发、资料汇总
- 马 **马子欣（25%）**：情绪数据集采集脚本创建、数据预处理与清洗、人脸数据采集与质量验证、PPT制作
- 杨 **杨逊源（25%）**：项目测试与优化、实训报告撰写、PPT制作

TECHNICAL ARCHITECTURE

技术架构

核心框架

飞桨（PaddlePaddle）3.2.0 GPU版

国产深度学习框架，支持GPU并行计算加速

关键技术

计算机视觉
OpenCV Haar级联分类器

深度学习
CNN、ResNet、数据增强

Web开发
Flask、HTML/CSS/JS

数据处理
numpy、scikit-learn

系统整体架构

数据层

自定义数据集
7类情绪：开心、悲伤、愤怒、惊讶、恐惧、厌恶、中性

数据规模
总计4052张图像，训练集2639张、测试集1413张

模型层

EmotionResNetSmall
基于ResNet结构的轻量化模型

模型参数
参数量约1.2M，文件大小42.6MB

应用层

图片上传识别
支持拖拽上传，实时识别情绪

摄像头实时识别
调用getUserMedia API，实时预览

TRAINING STEPS · MODULE 1-4

实训步骤：模块一

01 项目环境搭建与工具准备

2学时

- 创建Python虚拟环境，隔离系统环境
- 按顺序安装依赖库（使用清华镜像源）
- 验证飞桨GPU环境可用性

关键产出：环境配置文件requirements.txt、environment.yml

```
(.venv) PS D:\Administrator\Desktop\learn\face_project\face_emotion_datasets> python -c "import paddle; paddle.utils.run_check()"
>>
信息: 用提供的模式无法找到文件。
D:\Administrator\Desktop\learn\face_project\face_emotion\.venv\Lib\site-packages\paddle\utils\cpp_extension\extension_utils.py:718: UserWarning: No ccache found. Please be aware that recompiling all source files may be required. You can download and install ccache from: https://github.com/ccache/ccache/blob/master/doc/INSTALL.md
  warnings.warn(warning_message)
Running verify PaddlePaddle program ...
D:\Administrator\Desktop\learn\face_project\face_emotion\.venv\Lib\site-packages\paddle\pir\math_op_patch.py:219: UserWarning: Value do not have 'place' interface for pir graph mode, try not to use it. None will be returned.
  warnings.warn(
I0105 09:10:16.156168 26380 pir_interpreter.cc:1524] New Executor is Running ...
WARNING: Logging before InitGoogleLogging() is written to STDERR
W0105 09:10:16.156168 26380 gpu_resources.cc:114] Please NOTE: device: 0, GPU Compute Capability: 8.9, Driver API Version: 13.1, Runtime API Version: 12.6
I0105 09:10:16.160167 26380 pir_interpreter.cc:1547] pir interpreter is running by multi-thread mode ...
PaddlePaddle works well on 1 GPU.
PaddlePaddle is installed successfully! Let's start deep learning with PaddlePaddle now.
(.venv) PS D:\Administrator\Desktop\learn\face_project\face_emotion_datasets>
```

飞桨框架验证

requirements.txt U

requirements.txt

```
1  --index-url https://pypi.tuna.tsinghua.edu.cn/simple
2  --extra-index-url https://www.paddlepaddle.org.cn/packages/stable/cu126/
3
4  paddlepaddle-gpu==3.2.0
5  opencv-python==4.8.1.78
6  numpy==1.24.3
7  flask==2.3.3
8  pillow==10.0.1
9  scikit-learn==1.3.0
10 matplotlib==3.7.2
11
```

依赖库版本查验

TRAINING STEPS · MODULE 2

实训步骤：模块二

02 人脸数据采集与质量验证

4学时

- 基于OpenCV编写采集脚本，支持7类情绪视频录制与抽帧
- 组织数据采集，总计获取4052张样本
- 编写自动化脚本，实现人脸检测、清晰度评估等质量验证

关键产出：4052张高质量图像、自动化清洗脚本

```
==========================================================
情绪数据采集系统 - 主菜单    数据采集运行主菜单
==========================================================
1. 录制情绪视频
2. 从视频中抽帧
3. 查看目录结构
4. 查看数据统计
5. 初始化摄像头
0. 退出系统
==========================================================
请输入选择 (0-5):
```

```
总体统计:
  总图片数: 4052
  成功处理: 4052 (100.0%)
  处理失败: 0 (0.0%)

各情绪详细统计:
  angry      :  430/ 430 (100.0%)
  disgust    :  671/ 671 (100.0%)
  fear       :  578/ 578 (100.0%)
  happy      :  397/ 397 (100.0%)
  neutral    :  673/ 673 (100.0%)
  sad        :  661/ 661 (100.0%)
  surprise   :  642/ 642 (100.0%)

输出目录: ./emotion_dataset_enhanced
```

```
1. 面部清晰可见
2. 光线充足
3. 背景简洁
4. 表情自然明显
按任意键开始，按 'q' 退出预览

==========================================
情绪选择菜单
==========================================
请选择要录制的情绪类型:
  1. 开心 (happy)
  2. 悲伤 (sad)
  3. 愤怒 (angry)
  4. 惊讶 (surprise)
  5. 恐惧 (fear)
  6. 厌恶 (disgust)
  7. 中性 (neutral)
  0. 返回主菜单
==========================================
```

```
==========================================================
情绪数据采集系统 - 主菜单
==========================================================
1. 录制情绪视频
2. 从视频中抽帧
3. 查看目录结构
4. 查看数据统计
5. 初始化摄像头
0. 退出系统
==========================================================
请输入选择 (0-5): 2

==========================================================
视频抽帧菜单
==========================================================
请选择操作:
1. 抽取所有情绪的帧
2. 抽取特定情绪的帧
0. 返回主菜单
==========================================================
请输入选择 (0-2):
```

TRAINING STEPS · MODULE 3

实训步骤：模块三扩展

01 数据清洗与目录结构

• 原始采集图像经初步筛选后，通过自动化脚本结合人工审核，剔除模糊、侧脸、遮挡等无效样本，形成规范化目录结构，便于后续处理。




```
已处理 4000 张原始图片，生成 20000 张增强图片
已处理 4010 张原始图片，生成 20050 张增强图片
已处理 4020 张原始图片，生成 20100 张增强图片
已处理 4030 张原始图片，生成 20150 张增强图片
已处理 4040 张原始图片，生成 20200 张增强图片
已处理 4050 张原始图片，生成 20250 张增强图片

数据增强完成！
原始图片总数： 4052
生成的增强图片总数： 20260
增强后总图片数： 24312
```

运行结果

增强完成后，原4052张图像扩展至24312张，提升数据多样性。

02 数据增强实现

• 使用datastrong.py脚本，基于img库实施多层次数据增强，每张原始图像生成5张增强图像，有效扩充数据规模。增强策略包括几何变换（翻转、旋转、缩放）与色彩调整（亮度、对比度、饱和度等）。

1vediotodatasets.py　datastrong.py

```python
def create_augmentation_pipeline():
    """数据增强"""
    seq = iaa.Sequential( children: [
        # 仿射变换
        iaa.Affine(
            scale={"x": (0.8, 1.2), "y": (0.8, 1.2)},  # 随机缩放
            translate_percent={"x": (-0.2, 0.2), "y": (-0.2, 0.2)},  # 随机平移
            rotate=(-30, 30),  # 随机旋转
            shear=(-10, 10),  # 随机剪切
            order=[0, 1],  # 使用最近邻或双线性插值
            cval=(0, 255),  # 填充值范围
            mode='reflect'  # 边缘填充模式
        ),
        # 随机水平翻转
        iaa.Fliplr(0.5),
        # 随机垂直翻转
        iaa.Flipud(0.3),
        # 随机裁剪
        iaa.Sometimes(
            p 0.3,
            iaa.Crop(percent=(0, 0.1))
        ),
        # 随机对比度调整
        iaa.LinearContrast((0.8, 1.2)),
        # 随机亮度调整
        iaa.MultiplyBrightness((0.8, 1.2)),
        # 随机添加高斯噪声
        iaa.Sometimes(
            p 0.3,
            iaa.AdditiveGaussianNoise(loc=0, scale=(0.0, 0.05 * 255))
        ),
        # 随机模糊
        iaa.Sometimes(
            p 0.3,
            iaa.GaussianBlur(sigma=(0, 1.0))
        ),
        # 随机调整饱和度
```

对照片进行了仿射变换（旋转、平移、缩放、剪切）随机翻转、颜色调整（亮度、对比度、饱和度）添加噪声和模糊等数据增强操作。

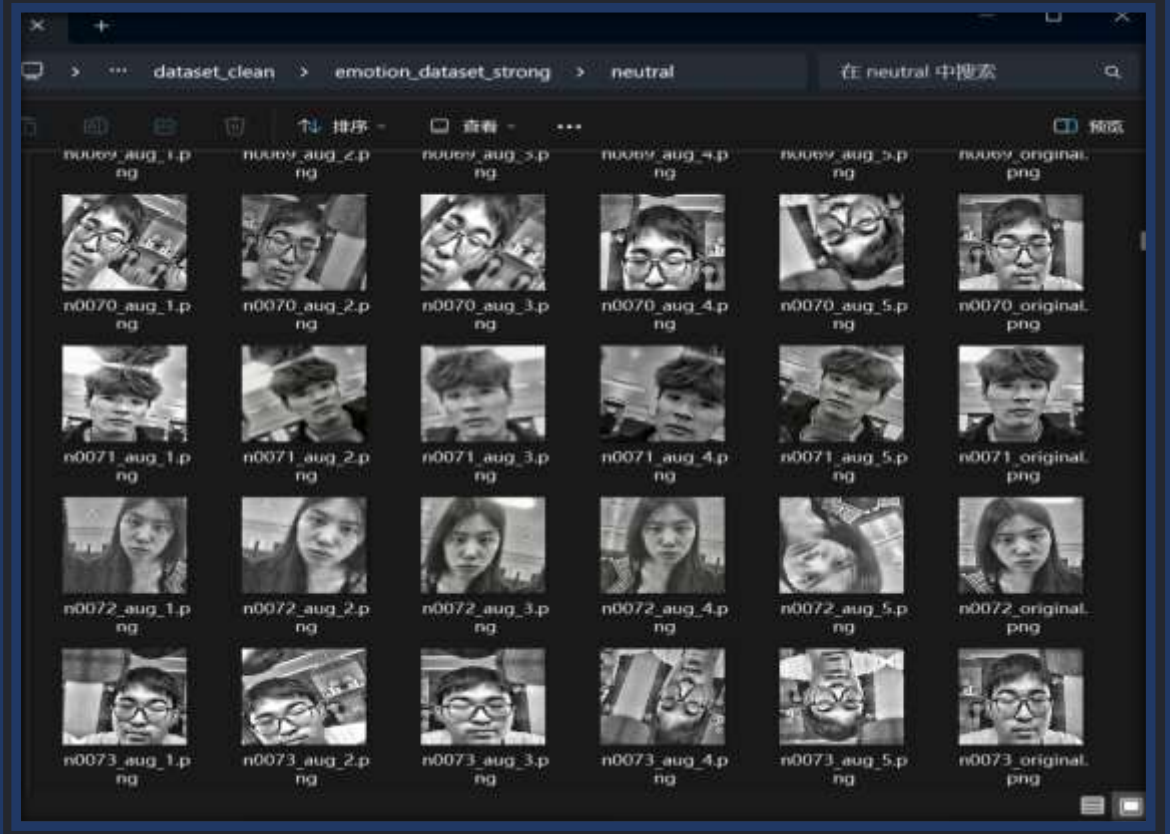
实训步骤：模块三扩展成果展示

03 增强效果可视化

- 增强后图像保留了原始表情特征，同时引入了合理的形态与光照变化。

下图展示了部分增强样本（以happy、neutral类别为例），每行包含1张原始图像与5张增强图像：





TRAINING STEPS · MODULE 3-4

实训步骤：模块三至模块四

03 数据预处理与数据集构建

4学时

- 执行灰度转换、像素归一化、维度扩展等预处理
- 按8:2比例划分训练集与测试集，序列化保存数据集

关键产出：emotion_dataset.pkl（含X_train, y_train, X_test, y_test）

04 CNN模型设计与飞桨实现

4学时

- 设计EmotionResNetSmall类（ResNet结构，含四个基本块）
- 验证模型结构，参数量约1.2M

关键产出：EmotionResNetSmall模型定义、模型结构验证报告

```
Administrator ❯❯ python -c "import data_collector as dc; print(
dc.extract_faces_dataset(src_dir='emotion_dataset', dst_dir='emotion
_dataset_faces'))"
{'total': 4052, 'saved': 2639, 'skipped': 1413, 'dst_dir': 'emotion_
dataset_faces'}
```

训练集 / 测试集划分结果

```python
class EmotionResNetSmall(nn.Layer):
    def __init__(self, num_classes=7):
        super().__init__()
        self.in_channels = 64

        self.stem = nn.Sequential(
            nn.Conv2D(1, 64, kernel_size=3, stride=1, padding=1, bias_attr=False),
            nn.BatchNorm2D(64),
            nn.ReLU(),
        )

        self.layer1 = self._make_layer(64, blocks=2, stride=1)
        self.layer2 = self._make_layer(128, blocks=2, stride=2)
        self.layer3 = self._make_layer(256, blocks=2, stride=2)
        self.layer4 = self._make_layer(512, blocks=2, stride=2)

        self.avgpool = nn.AdaptiveAvgPool2D((1, 1))
        self.dropout = nn.Dropout(0.4)
        self.fc = nn.Linear(512, num_classes)

    def _make_layer(self, out_channels, blocks, stride):
        layers = []
        layers.append(BasicBlock(self.in_channels, out_channels, stride=stride))
        self.in_channels = out_channels
        for _ in range(1, blocks):
            layers.append(BasicBlock(self.in_channels, out_channels, stride=1))
        return nn.Sequential(*layers)

    def forward(self, x):
        x = self.stem(x)
        x = self.layer1(x)
        x = self.layer2(x)
        x = self.layer3(x)
        x = self.layer4(x)
        x = self.avgpool(x)
        x = paddle.flatten(x, 1)
        x = self.dropout(x)
```

EmotionResNetSmall类定义代码

前四个模块累计耗时

环境搭建 → 数据采集 → 预处理 → 模型设计

14学时

约占整个实训周期的56%

TRAINING STEPS · MODULE 3-4

实训步骤：模块三至模块四

03 数据预处理与数据集构建

4学时

- 执行灰度转换、像素归一化、维度扩展等预处理
- 按8:2比例划分训练集与测试集，序列化保存数据集

关键产出：emotion_dataset.pkl（含X_train, y_train, X_test, y_test）

04 CNN模型设计与飞桨实现

4学时

- 设计EmotionResNetSmall类（ResNet结构，含四个基本块）
- 验证模型结构，参数量约1.2M

关键产出：EmotionResNetSmall模型定义、模型结构验证报告

```
Administrator ❯❯ python -c "import data_collector as dc; print(
dc.extract_faces_dataset(src_dir='emotion_dataset', dst_dir='emotion
_dataset_faces'))"
{'total': 4052, 'saved': 2639, 'skipped': 1413, 'dst_dir': 'emotion_
dataset_faces'}
```

训练集 / 测试集划分结果

```python
class EmotionResNetSmall(nn.Layer):
    def __init__(self, num_classes=7):
        super().__init__()
        self.in_channels = 64

        self.stem = nn.Sequential(
            nn.Conv2D(1, 64, kernel_size=3, stride=1, padding=1, bias_attr=False),
            nn.BatchNorm2D(64),
            nn.ReLU(),
        )

        self.layer1 = self._make_layer(64, blocks=2, stride=1)
        self.layer2 = self._make_layer(128, blocks=2, stride=2)
        self.layer3 = self._make_layer(256, blocks=2, stride=2)
        self.layer4 = self._make_layer(512, blocks=2, stride=2)

        self.avgpool = nn.AdaptiveAvgPool2D((1, 1))
        self.dropout = nn.Dropout(0.4)
        self.fc = nn.Linear(512, num_classes)

    def _make_layer(self, out_channels, blocks, stride):
        layers = []
        layers.append(BasicBlock(self.in_channels, out_channels, stride=stride))
        self.in_channels = out_channels
        for _ in range(1, blocks):
            layers.append(BasicBlock(self.in_channels, out_channels, stride=1))
        return nn.Sequential(*layers)

    def forward(self, x):
        x = self.stem(x)
        x = self.layer1(x)
        x = self.layer2(x)
        x = self.layer3(x)
        x = self.layer4(x)
        x = self.avgpool(x)
        x = paddle.flatten(x, 1)
        x = self.dropout(x)
```

EmotionResNetSmall类定义代码

前四个模块累计耗时

环境搭建 → 数据采集 → 预处理 → 模型设计

14学时

约占整个实训周期的56%

TRAINING STEPS · MODULE 5-8

实训步骤：模块五

05 模型训练与超参数优化

4学时

- 配置训练参数（batch_size=32、epochs=50等）
- 采用AdamW优化器、余弦退火学习率调度
- 实施数据增强与类别均衡策略，训练并保存最佳模型

测试准确率	最佳准确率	训练耗时
99.53%	99.69%	~45min

```
Epoch [20/20] Step 90/165 Loss 0.4520 Acc 1.0000 6.30 it/s ETA 12s
Epoch [20/20] Step 100/165 Loss 0.4520 Acc 1.0000 6.30 it/s ETA 10s
Epoch [20/20] Step 110/165 Loss 0.4520 Acc 1.0000 6.30 it/s ETA 9s
Epoch [20/20] Step 120/165 Loss 0.4520 Acc 1.0000 6.31 it/s ETA 7s
Epoch [20/20] Step 130/165 Loss 0.4520 Acc 1.0000 6.30 it/s ETA 6s
Epoch [20/20] Step 140/165 Loss 0.4521 Acc 1.0000 6.26 it/s ETA 4s
Epoch [20/20] Step 150/165 Loss 0.4520 Acc 1.0000 6.25 it/s ETA 2s
Epoch [20/20] Step 160/165 Loss 0.4520 Acc 1.0000 6.24 it/s ETA 1s
Epoch [20/20] Step 165/165 Loss 0.4523 Acc 1.0000 6.27 it/s ETA 0s
Epoch [20/20]
Train Loss: 0.4523, Train Acc: 1.0000
Test Loss: 0.4582, Test Acc: 0.9953
--------------------------------------------------
训练完成！最佳测试准确率: 0.9969
```

训练轮次（Epoch）：

- 共 20

训练总步骤：

- 165 步

训练损失（Train Loss）：

- 0.4523

训练准确率（Train Acc）：

- 1.0000（即 100%）

测试损失（Test Loss）：

- 0.4582

测试准确率（Test Acc）：

- 0.9953

训练状态：训练完成

- 最佳测试准确率：0.9969

模块五至八	平均响应时间	代码质量	Bug修复率
11学时	<1.5s	95%+	100%
训练·开发·集成·测试	端到端优化	模块化·规范化	12个Bug已修复

TRAINING STEPS · MODULE 5-8

实训步骤：模块六

Web前端界面预览截图



上传图片
展示识别情绪结果



摄像头实时识别功能
可以实时识别表情



06 **Web前端界面开发**

3学时

- 用CSS美化界面，JavaScript实现前后端交互

- 编写HTML页面实现拖拽上传、图片预览、摄像头控制功能

- 用CSS美化界面，JavaScript实现前后端交互

拖拽上传	实时预览	响应式
✓	✓	✓

TRAINING STEPS · MODULE 5-8

实训步骤：模块七

07

后端接口开发与模型集成

2学时

- 基于Flask

搭建Web服务，

定义/predict、/capture接口

- 加载训练好的模型，

集成图像预处理流程

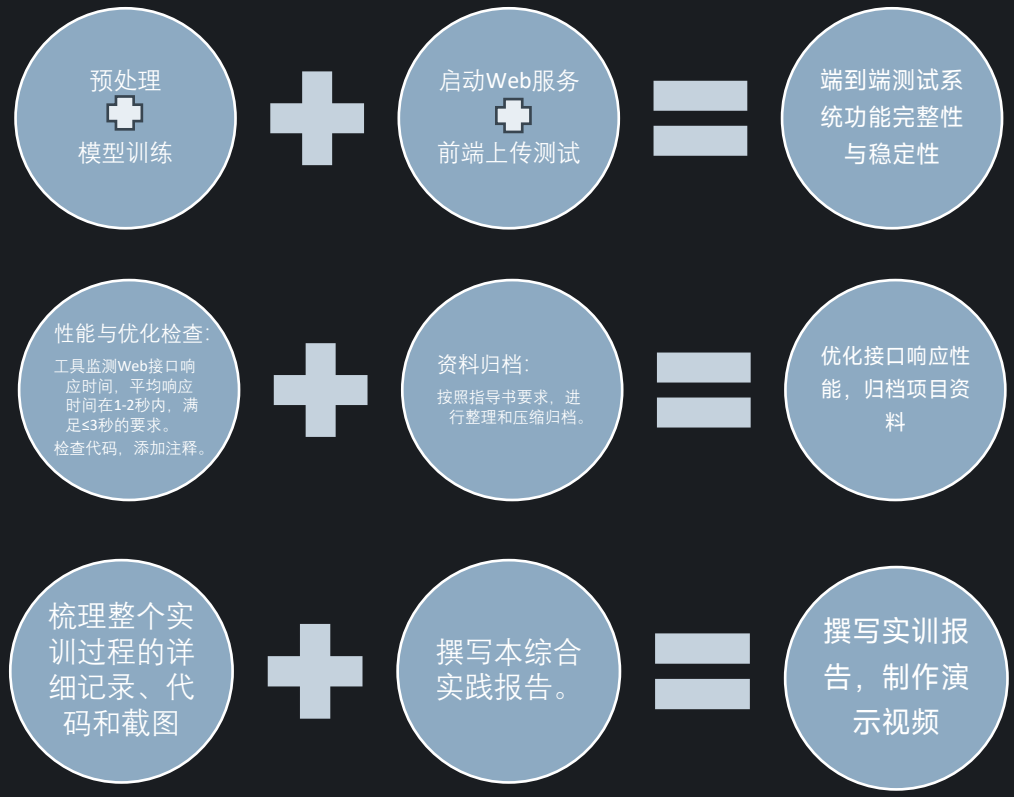
关键产出：RESTful API、模型推理服务、前后端联调

```
Administrator )) python web_server.py
信息：用提供的模式无法找到文件。
D:\Administrator\Desktop\learn\face_project\face_emotion\.venv\Lib\site-packages\paddle\utils\cpp_extension\extension_utils.py:718: UserWarning: No ccache found. Please be aware that recompiling all source files may be required. You can download and install ccache from: https://github.com/ccache/ccache/blob/master/doc/INSTALL.md
  warnings.warn(warning_message)
 * Serving Flask app 'web_server'
 * Debug mode: on
[2026-01-05 14:26:36,031] [    INFO] _internal.py:97 - WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5001
[2026-01-05 14:26:36,031] [    INFO] _internal.py:97 - Press CTRL+C to quit
[2026-01-05 14:26:36,409] [    INFO] _internal.py:97 - 127.0.0.1 - - [05/Jan/2026 14:26:36] "GET / HTTP/1.1" 200 -
[2026-01-05 14:39:25,770] [    INFO] _internal.py:97 - 127.0.0.1 - - [05/Jan/2026 14:39:25] "POST /predict HTTP/1.1" 200 -
WARNING: Logging before InitGoogleLogging() is written to STDERR
W0105 14:39:42.250077 29420 gpu_resources.cc:114] Please NOTE: device: 0, GPU Compute Capability: 8.9, Driver API Version: 13.1, Runtime API Version: 12.6
```

模块五至八	平均响应时间	代码质量	Bug修复率
11学时	<1.5s	95%+	100%
训练·开发·集成·测试	端到端优化	模块化·规范化	12个Bug已修复

TRAINING STEPS · MODULE 5-8

实训步骤：模块八



08 项目测试、优化与报告撰写

2学时

- 端到端测试系统功能完整性与稳定性
- 优化接口响应性能，归档项目资料
- 撰写实训报告，制作演示视频

关键产出：测试报告、项目文档、演示视频、总结PPT

模块五至八	平均响应时间	代码质量	Bug修复率
11学时	<1.5s	95%+	100%
训练·开发·集成·测试	端到端优化	模块化·规范化	12个Bug已修复

TRAINING RESULTS

实训成果

数据集成果

情绪类别	7类
总图像数	4052张
训练集	2639张
测试集	1413张
图像尺寸	48×48

模型性能成果

测试准确率	99.53%
最佳测试准确率	99.69%
训练准确率	99.87%
模型文件大小	42.6MB
模型参数量	1.2M

系统功能成果

Web服务
http://localhost:5001

识别模式
图片上传识别、摄像头实时识别

响应时间
平均响应时间 < 1.5秒

界面设计
前端界面友好，响应式布局

```
Administrator ❯❯ python -c "import data_collector as dc; print(dc.extract_faces_dataset(src_dir='emotion_dataset', dst_dir='emotion_dataset_faces'))"
{'total': 4052, 'saved': 2639, 'skipped': 1413, 'dst_dir': 'emotion_dataset_faces'}
```

训练集 / 测试集划分结果

best_emotion_model.pdparams	2026/1/5 13:58	PDPARAMS 文件	43,684 KB
data_collector.py	2026/1/5 13:45	Python.File	10 KB
emotion_dataset.pkl	2025/12/28 23:59	PKL 文件	276,946 KB

模型文件大小

TRAINING RESULTS

实训成果

验证案例

中性情绪

识别结果 neutral

置信度 78.09%



悲伤情绪

识别结果 sad

置信度 90.63%



惊讶情绪

识别结果 surprise

置信度 90.41%



PROBLEMS & LIMITATIONS

问题与不足

遇到的关键问题

1. GPU环境配置失败

问题：飞桨GPU版本安装后验证失败，CUDA与cuDNN版本不匹配

解决：重装CUDA 12.6 + cuDNN 8，确保版本兼容，配置环境变量后验证通过

2. 模型训练初期严重过拟合

问题：训练准确率高但验证准确率低，模型泛化能力差

解决：引入数据增强 + 标签平滑 + Dropout，增加训练数据多样性，最终测试准确率达99.53%

3. Web前端上传图片后后端预测报错

问题：图像解码方式不当，Base64编码数据传到后端后解码失败

解决：修正图片解码逻辑，统一前后端编码格式，添加数据校验和异常处理

4. 数据采集时光线不均匀导致样本质量差

问题：采集的图像光照条件不一致，部分图像过亮或过暗

解决：调整采集环境保证光照均匀，后期增加自动筛选过滤低质量图像

项目存在的不足

1. **模型实用化与轻量化不足**
   当前模型不适配移动端/嵌入式环境，推理延迟有待优化
2. **系统鲁棒性与泛化能力有限**
   对极端场景（遮挡、侧脸、极端光照）适应性差
3. **数据集场景单一**
   缺乏多场景、多光照、多人种数据，多样性不足
4. **工程实现可优化**
   部分代码模块化、配置化程度不足，可维护性待提升

经验总结：通过解决这些问题，我们深刻理解了深度学习项目开发的全流程挑战，积累了宝贵的工程实践经验，提升了解决问题的能力和技术攻关经验。认识到**数据质量、环境配置、模型优化、工程实现**是AI项目成功的四大支柱。

LEARNING SUMMARY

学习总结

核心收获

技术能力

- 掌握飞桨框架高级功能，从环境配置到部署上线
- 熟练运用CNN卷积神经网络进行图像分类任务
- 掌握OpenCV进行图像处理和Haar级联分类器人脸检测
- 掌握Flask Web框架开发RESTful API

工程思维

- 建立从数据到部署的系统化开发思维
- 理解AI项目完整生命周期，重视数据质量
- 掌握模块化设计和代码规范，提升可维护性
- 理解性能优化的重要性，全链路优化

团队协作与自主学习

- 提升分工协作效率，强化团队沟通能力
- 增强自主学习能力，通过查阅文档解决问题
- 提升技术文档编写能力，编写清晰API文档
- 掌握Git版本管理，进行协作开发

未来展望

1. **模型优化**
   尝试MobileNet结构、量化压缩等轻量化方案，实现移动端实时推理
2. **功能扩展**
   增加语音、文本多模态识别，提升情绪识别的准确性和鲁棒性
3. **场景延伸**
   开发移动端APP，扩大应用覆盖面，应用于教育、医疗等实际场景

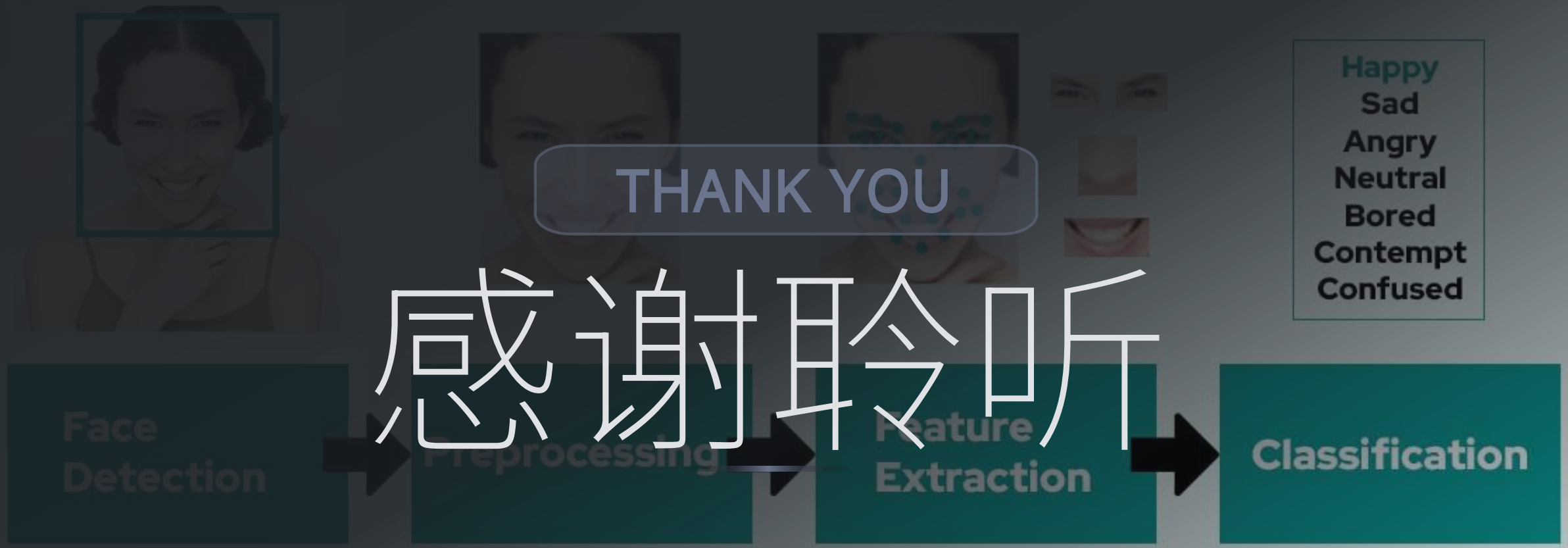
迭代计划

短期目标（3-6个月）

- 优化模型结构，提升推理速度30%
- 扩充数据集，增加多样本
- 完成模型量化，适配边缘设备

长期愿景（1-2年）

- 开发跨平台情绪识别SDK
- 探索多模态情绪融合技术
- 应用于教育、医疗等实际场景



THANK YOU

感谢聆听